

Задание 3. Протокол Диффи – Хеллмана

Протокол Диффи-Хеллмана позволяет двум сторонам обмениваться данными через незащищённый канал для выработки общего секретного ключа.

Основные шаги:

1. Выбираются общие параметры: большое простое число p и генератор g .
2. Клиент и сервер генерируют свои приватные ключи a и b соответственно.
3. Они вычисляют публичные ключи:

$$A = g^a \mod p, \quad B = g^b \mod p$$

4. Публичные ключи обмениваются между сторонами.
5. Вырабатывается общий секрет:

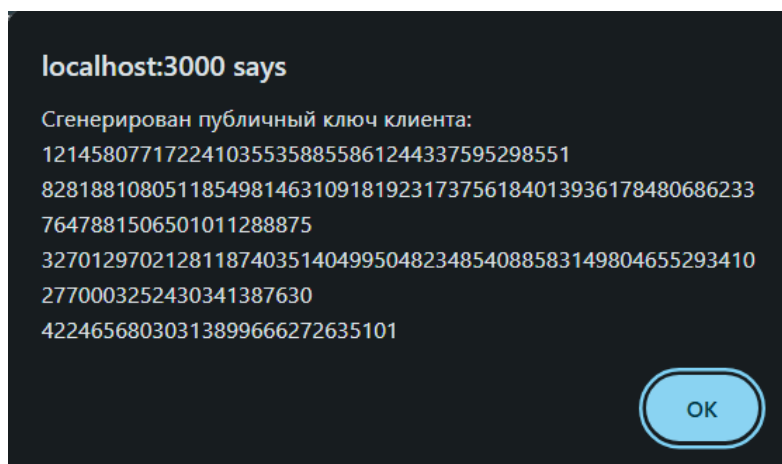
$$s = B^a \mod p = A^b \mod p$$

Реализация:

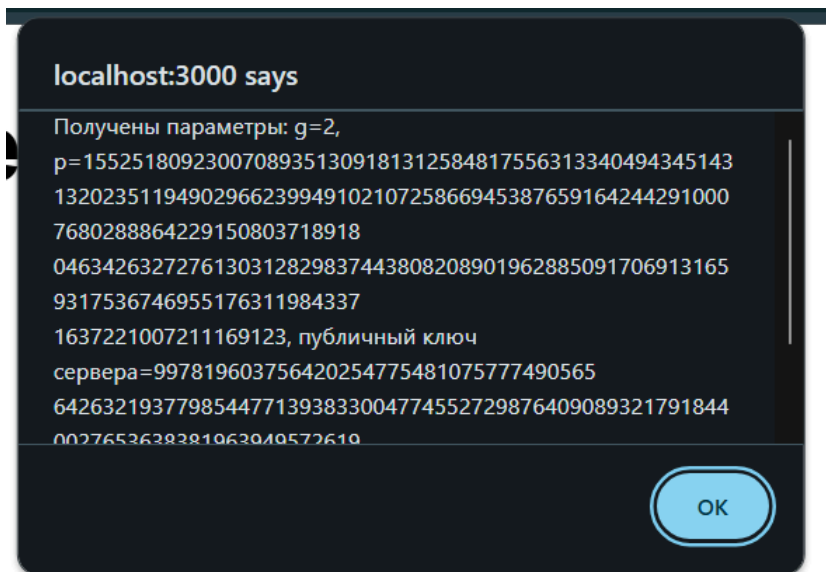
Клиент:

Клиентская часть реализована на HTML/JavaScript с использованием модуля `crypto-js` для работы с хэшами. Основные этапы:

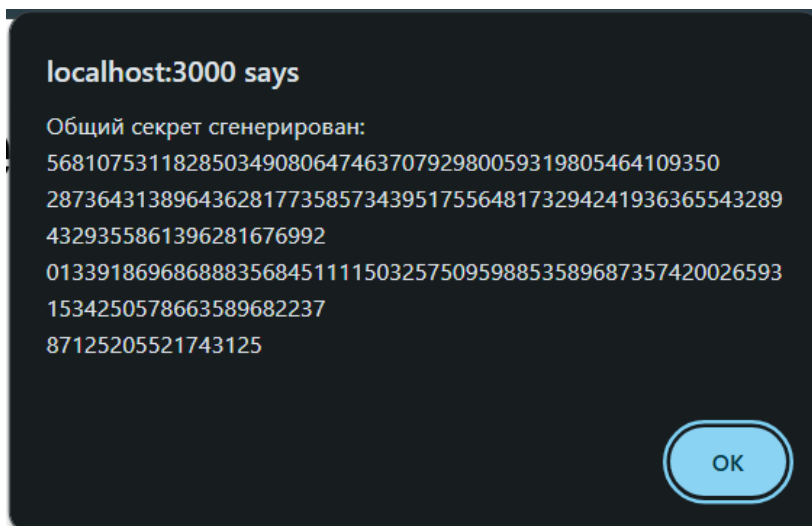
1. Генерация приватного ключа клиента через криптографический генератор случайных чисел.
2. Вычисление публичного ключа клиента.



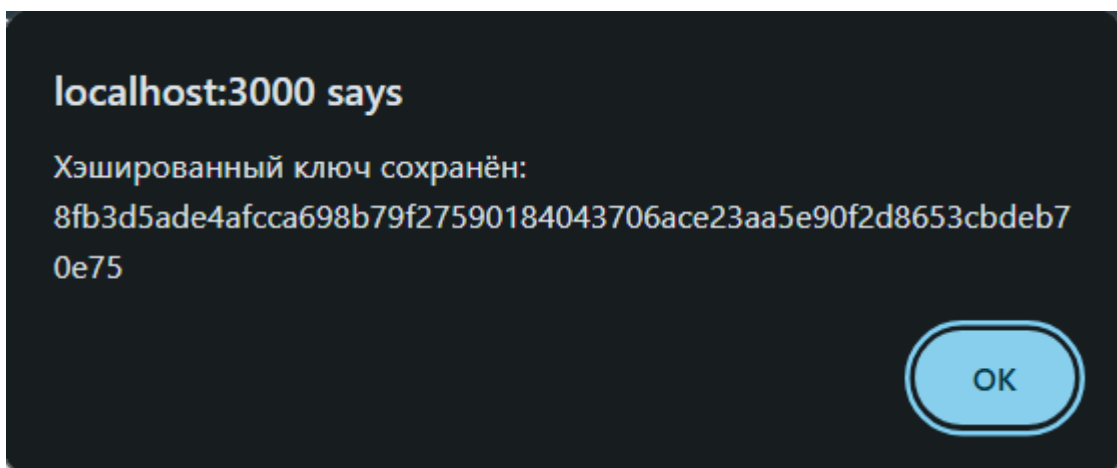
3. Обмен публичными ключами с сервером.



4. Генерация общего секрета.

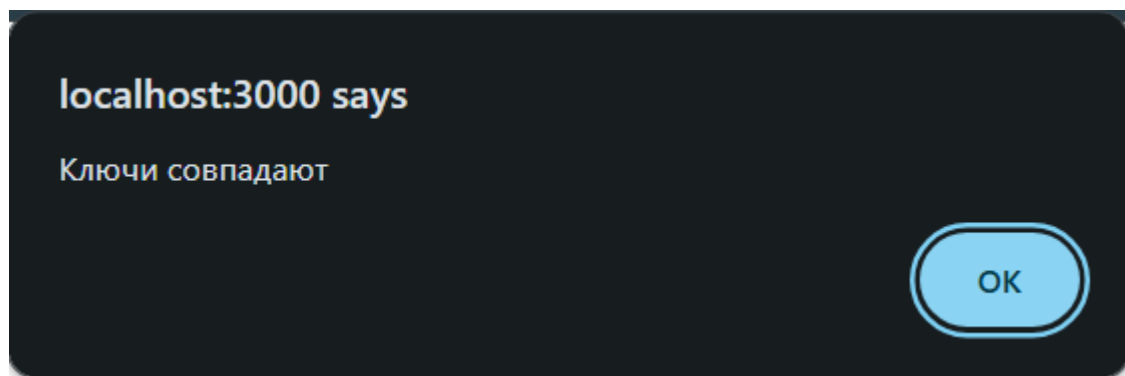


5. Хэширование общего секрета с использованием соли, полученной от сервера.



6. Проверка корректности общего секрета через сервер.

```
[SERVER] Полученный хешированный ключ: 8fb3d5ade4afcca698b79f27590184043706ace23aa5e90f2d8653cbdeb70e75  
[SERVER] Ожидаемый хеш: 8fb3d5ade4afcca698b79f27590184043706ace23aa5e90f2d8653cbdeb70e75
```



Код содержит функции для:

- Возведения в степень по модулю `modPow`.
- Генерации случайного приватного ключа `generatePrivateKey`.

Сервер:

Серверная часть реализована на Node.js с использованием фреймворка Express.

Основные задачи:

1. Хранение параметров p , g , и приватного ключа сервера.
2. Генерация публичного ключа сервера.

```
[SERVER] Параметры g: 2, p: 15525180923007089351309181312584817556313340494345143132023511949029662  
27613031282983744380820890196288509170691316593175367469551763119843371637221007211169123  
[SERVER] Публичный ключ сервера: 99781960375642025477548107577490565642632193779854477139383300477  
992708167049667024529532635019943306805538638524133458912908636691406347851592203542118204997
```

3. Приём публичного ключа клиента и вычисление общего секрета.

```
[SERVER] Общий секрет для пользователя user: 152433543199292546361159421988121402545523130468017776315  
045233563150381029322020357774246953306329597441106550538465993862175517051219202289940341040969033411
```

4. Хранение общего секрета и соли для каждого пользователя.
5. Проверка хэша общего секрета.

```
app.post('/check', (req, res) => {
  const { username, HasedKey } = req.body;

  console.log(`[SERVER] Полученный хешированный ключ: ${HasedKey}`);

  const users = loadUsers();
  const user = users[username];

  if (!user) {
    return res.status(400).send('Пользователь не найден');
  }

  const expectedHash = crypto.createHash('sha256').update(user.sharedSecret + salt).digest('hex');
  console.log(`[SERVER] Ожидаемый хеш: ${expectedHash}`);
  if (expectedHash === HasedKey) {
    res.send('Ключи совпадают');
  } else {
    res.status(400).send('Ключи не совпадают');
  }
});
```

Взаимодействие клиента и сервера:

1. **Запрос параметров:** Клиент запрашивает g , p , и публичный ключ сервера.
 2. **Отправка публичного ключа клиента:** Клиент отправляет свой публичный ключ серверу.
 3. **Генерация общего секрета:** Сервер вычисляет общий секрет и отправляет соль клиенту.
 4. **Хэширование секрета:** Клиент и сервер независимо хэшируют общий секрет с солью.
 5. **Проверка:** Клиент отправляет хэшированный ключ серверу для проверки.
-